T3000 API & Protocol

Variables · built Nov 13, 2026

Variables

Calculated variables, setpoints, and derived values.

Overview

Variables store calculated values, setpoints, intermediate results, and system status flags. They're used for complex control logic, data manipulation, and system coordination.

Variable Types

Analog Variables (AV)

- Setpoints (temperature, pressure)
- Calculated values (averages, differentials)
- Timers and counters
- Mathematical results

Binary Variables (BV)

- Status flags
- Enable/disable switches
- Alarm conditions
- Mode selections

Common Uses

- **Temperature Setpoints**: Heating/cooling targets
- **Differential Calculations**: Supply - Return temperature
- **Runtime Accumulators**: Equipment operating hours
- **Status Flags**: System mode indicators
- **Energy Calculations**: kW, BTU/hr

Configuration

Variables can be:

- **Read-only**: Calculated by programs
- **Writable**: User-adjustable setpoints
- **Commandable**: Controlled by programs or operators

Best Practices

- Name clearly (e.g., Zone_Temp_SP)
- Document calculations
- Set appropriate limits
- Use consistent units

Next Steps

- Programs - Control logic
- PID Loops - PID control

Variables

Variable Management API

Reading and Writing Variables

TYPESCRIPT

```typescript
// Read variable value
const response = await fetch(
  `http://localhost:9103/api/devices/389001/points/VAR1`
);
const data = await response.json();

// Write variable value
await fetch(`http://localhost:9103/api/devices/389001/points/VAR1`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ value: 72.5 })
});

// BACnet analog-value object
await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'analog-value',
  objectInstance: 1,
  property: 'present-value',
  value: 72.5
});
```

Variable Calculations

TYPESCRIPT

```typescript
class VariableCalculator {
  async updateCalculatedVariables(deviceId: number) {
    // Read source values
    const supplyTemp = await this.readPoint(deviceId, 'IN1');
    const returnTemp = await this.readPoint(deviceId, 'IN2');

    // Calculate differential
    const differential = supplyTemp - returnTemp;
    await this.writeVariable(deviceId, 'VAR_Diff', differential);

    // Calculate average
    const average = (supplyTemp + returnTemp) / 2;
    await this.writeVariable(deviceId, 'VAR_Avg', average);

    // Runtime accumulator
    const fanStatus = await this.readPoint(deviceId, 'BO1');
    if (fanStatus) {
      const runtime = await this.readVariable(deviceId, 'VAR_Runtime');
      await this.writeVariable(deviceId, 'VAR_Runtime', runtime + 1);
    }
  }
}
```

Setpoint Management

TYPESCRIPT

```typescript
interface SetpointSchedule {
  occupiedSP: number;
  unoccupiedSP: number;
  schedule: {
    start: string;  // "07:00"
    end: string;    // "18:00"
  };
}

class SetpointManager {
  async updateSetpoint(deviceId: number, config: SetpointSchedule) {
    const now = new Date();
    const hour = now.getHours();
    const minute = now.getMinutes();
    const currentTime = `${hour.toString().padStart(2, '0')}:${minute.toString().padStar

    const isOccupied = currentTime >= config.schedule.start &&
                       currentTime <= config.schedule.end;

    const setpoint = isOccupied ? config.occupiedSP : config.unoccupiedSP;

    await this.writeVariable(deviceId, 'VAR_Setpoint', setpoint);
  }
}
```

Mathematical Operations

Formula Evaluation

TYPESCRIPT

```typescript
class FormulaEngine {
  evaluate(formula: string, variables: Record<string, number>): number {
    // Simple expression evaluator
    // Replace variable names with values
    let expression = formula;
    for (const [name, value] of Object.entries(variables)) {
      expression = expression.replace(new RegExp(name, 'g'), value.toString());
    }

    // Evaluate (Note: Use safe-eval in production)
    return eval(expression);
  }
}

// Example: Calculate enthalpy
const formula = '(0.24 * TEMP) + (HUM/100) * (1061 + 0.444 * TEMP)';
const engine = new FormulaEngine();

const enthalpy = engine.evaluate(formula, {
  TEMP: 75,   // °F
  HUM: 50     // %RH
});

console.log(`Enthalpy: ${enthalpy.toFixed(2)} BTU/lb`);
```

Statistical Functions

TYPESCRIPT

```typescript
class Statistics {
  static average(values: number[]): number {
    return values.reduce((a, b) => a + b, 0) / values.length;
  }

  static min(values: number[]): number {
    return Math.min(...values);
  }

  static max(values: number[]): number {
    return Math.max(...values);
  }

  static stdDev(values: number[]): number {
    const avg = this.average(values);
    const squareDiffs = values.map(v => Math.pow(v - avg, 2));
    const avgSquareDiff = this.average(squareDiffs);
    return Math.sqrt(avgSquareDiff);
  }
}

// Calculate zone temperature statistics
const temps = [72.1, 72.5, 71.8, 72.3, 72.0];

const stats = {
  avg: Statistics.average(temps),
  min: Statistics.min(temps),
  max: Statistics.max(temps),
  stdDev: Statistics.stdDev(temps)
};
```

Variable Persistence

Database Storage

SQL

```sql
CREATE TABLE variables (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER NOT NULL,
  variable_id TEXT NOT NULL,
  value REAL NOT NULL,
  units TEXT,
  writable BOOLEAN DEFAULT 1,
  updated_at INTEGER NOT NULL,
  UNIQUE(device_id, variable_id),
  FOREIGN KEY (device_id) REFERENCES devices(id)
);

CREATE INDEX idx_variables_device
  ON variables(device_id, variable_id);
```

Variable Caching

TYPESCRIPT

```typescript
class VariableCache {
  private cache = new Map<string, {value: number, timestamp: number}>();
  private ttl = 5000; // 5 seconds

  async get(deviceId: number, varId: string): Promise<number> {
    const key = `${deviceId}:${varId}`;
    const cached = this.cache.get(key);

    if (cached && Date.now() - cached.timestamp < this.ttl) {
      return cached.value;
    }

    // Cache miss - fetch from device
    const value = await this.fetchFromDevice(deviceId, varId);
    this.cache.set(key, { value, timestamp: Date.now() });

    return value;
  }

  set(deviceId: number, varId: string, value: number) {
    const key = `${deviceId}:${varId}`;
    this.cache.set(key, { value, timestamp: Date.now() });
  }
}
```

Next Steps

- REST API Reference
- Programs
- Inputs